

Universitatea
Politehnica
București



Facultatea de
Automatică și
Calculatoare



Catedra de
Calculatoare

Laborator 3
Structuri de control și execuții ciclice

Autori

Conf. Dr. Ing. Alexandru Boicea
As. Drd. Ing. Ciprian-Octavian Truică



Cuprins

- Instrucțiunea IF ... THEN
- Instrucțiunea IF ... THEN ... ELSE
- Instrucțiunea IF ... THEN ... ELSIF ... ELSE
- Instrucțiunea CASE
- Instrucțiunea LOOP
- Instrucțiunea WHILE
- Instrucțiunea FOR



Prezentare

- Limbajul PL/SQL combină puterea de procesare a datelor, specifică limbajelor procedurale, cu puterea de manipulare a datelor, specifică instrucțiunilor;
- Structurile de control reprezintă cea mai importantă completare adusă de PL/SQL limbajului SQL prin introducerea instrucțiunilor de control condiționale, iterative, secvențiale și necondiționale;
- Toate structurile acceptă în clauzele lor instrucțiuni imbricate.

Instrucțiunea IF ... THEN

- Sintaxa logică:

  IF conditions

  THEN statements;

  END IF;

- **conditions** – reprezintă condițiile cerute pentru execuția instrucțiunilor;
- **statements** – reprezintă secvența de instrucțiuni condiționale;
- Instrucțiunile care urmează după THEN se execută numai dacă toate condițiile din **conditions** sunt îndeplinite.

Instrucțiunea IF ... THEN

- Ex. 1. Pentru un angajat cu funcția ‘Manager’ care nu primește comision să se modifice comisionul cu 10% din salariul său. Testați cu id-ul 7698.

```
set serveroutput on;
declare
   numeAngajat varchar2(20);
   comAngajat number;
   functie varchar2(15);
   ecuson emp.empno%type;
```



Instrucțiunea IF ... THEN

```
begin
   ecuson := &idAngajat;
   select ename, job, comm into numeAngajat, functie, comAngajat
     from emp
     where empno = ecuson;
   if lower(functie) = 'manager' and (comAngajat = 0 or comAngajat is null)
   then
     update emp set comm = 0.1*sal where empno = ecuson;
     dbms_output.put_line('Comisionul lui '||
       numeAngajat||' a fost modificat!');
   end if;
   exception
     when no_data_found then
       dbms_output.put_line('Nu exista angajat cu empno = '||ecuson);
end;
```

Instrucțiunea IF ... THEN ... ELSE

- Sintaxa acestei instrucțiuni este:

  IF conditions

  THEN statements_1;

  ELSE statements_2;

  END IF;

- **conditions** – reprezintă condițiile cerute pentru execuția **statements_1**
- **statements_1** – reprezintă secvența de instrucțiuni atunci când **conditions** este TRUE
- **statements_2** – reprezintă secvența de instrucțiuni atunci când **conditions** este FALSE

Instrucțiunea IF ... THEN ... ELSE

- Ex. 2. Să se acorde un comision de 10% din salariu angajaților care au o vechime de cel puțin 32 de ani în companie. (testati cu 7876 si 7369)

```
set serveroutput on;
declare
    numeAngajat varchar2(30);
    ecuson emp.empno%type;
    dataAngajare date;
```



Instrucțiunea IF ... THEN ... ELSE

```
begin
   ecuson := &idAng;
   select ename, hiredate into numeAngajat, dataAngajare
     from emp where empno = ecuson;
   if dataAngajare < add_months(sysdate, -384)
   then
     update emp set comm = 0.1 * sal where empno=ecuson;
     dbms_output.put_line('Angajatul '||numeAngajat||' a primit comision!');
   else
     dbms_output.put_line('Angajatul nu indeplineste conditia!');
   end if;
   exception
     when no_data_found then
       dbms_output.put_line('Nu exista angajat cu ecusonul='||ecuson);
end;
```



Instrucțiunea IF ... THEN ... ELSIF ... ELSE

- Sintaxa instrucțiunii este:

```
IF conditions_1
        THEN statements_1;
ELSIF conditions_2
        THEN statements_2;
...
[ELSE statements_k;]
END IF;
```



Instrucțiunea IF ... THEN ... ELSIF ... ELSE

- **conditions_1** – reprezintă setul de condiții cerute pentru execuția **statements_1**
- **statements_1** – reprezintă secvența de instrucțiuni executate atunci când **conditions_1** = TRUE
- **conditions_2** – reprezintă setul de condiții cerute pentru execuția **statements_2**
- **statements_2** – reprezintă secvența de instrucțiuni executate atunci când **conditions_2** = TRUE
- **statements_k** – reprezintă secvența de instrucțiuni executate dacă niciuna dintre **conditions_1..k-1** nu sunt verificate



Instrucțiunea IF ... THEN ... ELSIF ... ELSE

- Verificarea seturilor de condiții se face secvențial, în ordinea specificată și fiecare set poate conține condiții multiple;
- Se acceptă mai multe clause **ELSIF** dar o singură clasuză **ELSE;**
- Clauza **ELSE** este opțională, aceasta poate să lipsească din construcția instrucțiunii.

Instrucțiunea IF ... THEN ... ELSIF ... ELSE

- Ex. 3. Să se acorde un comision de 10% din salariu, conducerii companiei care nu a primit comision, dar președintele să nu primească comision.

```
set serveroutput on;
declare
   numeAngajat varchar2(20);
   comAngajat number;
   functie varchar2(15);
   ecuson emp.empno%type;
```

Instrucțiunea IF ... THEN ... ELSIF ... ELSE

```
begin
  ecuson := &idAngajat;
  select ename, job, comm into numeAngajat, functie, comAngajat
    from emp
    where empno = ecuson;
  if lower(functie) = 'president'
  then
    dbms_output.put_line('Presedintele nu primeste comision!');
  elsif lower(functie) = 'manager' and (comAngajat = 0 or comAngajat is null)
  then
    update emp set comm = 0.1*sal where empno = ecuson;
    dbms_output.put_line('Comisionul lui '||
      numeAngajat||' a fost modificat!');
  else
    dbms_output.put_line('Angajatul nu face parte din conducere!');
  end if;
  exception
    when no_data_found then
      dbms_output.put_line('Nu exista angajat cu empno = '||ecuson);
end;
```



Instrucțiunea CASE

- Sintaxa instrucțiunii:

  [<<label_name>>]

  CASE expresion

  WHEN value1 THEN statements_1;

  WHEN value2 THEN statements_2;

  ...

  [ELSE statements_k;]

  END CASE [label_name];

- **expresion** – reprezintă expresia care se va evalua
- **statements_1** – reprezintă secvența de instrucțiuni executate în cazul în care **expresion = value1**
- **statements_2** – reprezintă secvența de instrucțiuni executate în cazul în care **expresion = value2**
- **statements_k** – reprezintă secvența de instrucțiuni executate dacă **expresion <> value1, value2, ...**



Instrucțiunea CASE

- Ex. 4. Să se calculeze impozitul pe venit al unui angajat, după algoritmul:
  - Se calculează venitul = sal + comm;
  - Din tabela salgrade se află gradul de salarizare;
  - În funcție de gradul de salarizare se aplică un impozit de 10%, 15%, 20% sau 25% din venit;
  - Dacă venitul este în afara grilei se aplică un impozit de 30% din venit;
  - Se afișează numele, venitul și impozitul angajatului.



Instrucțiunea CASE

```
set serveroutput on;
declare
   numeAngajat varchar2(20);
   venit number;
   gradSalariu integer;
   impozit number;
   ecuson emp.empno%type;
```

Instrucțiunea CASE

```
begin
  ecuson :=&idAngajat;
  select ename, sal+nvl(comm, 0) into numeAngajat, venit
    from emp where empno = ecuson;
  select grade into gradSalariu from salgrade
    where losal<venit and venit<=hisal;

  <<mycase>> -- exemplu de folosire eticheta(nu este obligatorie)
  case gradSalariu
    when 1 then impozit := venit*0.10;
    when 2 then impozit := venit*0.15;
    when 3 then impozit := venit*0.20;
    when 4 then impozit := venit*0.25;
  else
    impozit := venit*0.30;
  end case mycase;

  dbms_output.put_line('Angajatul '||numeAngajat||
    ' are venitul='||venit||' si un impozit='||impozit);
exception
  when no_data_found then
    dbms_output.put_line('Nu exista angajat cu ecusonul='||ecuson);
end;
```



Instrucțiunea CASE

- PL/SQL pune la dispoziția dezvoltatorilor ***searched CASE ;***
- Sintaxa instrucțiunii ***searched* case** este:

[<<label_name>>]

CASE

WHEN search_condition1 THEN sequence_of_statements1;

WHEN search_condition2 THEN sequence_of_statements2;

...

WHEN search_conditionN THEN sequence_of_statementsN;

[ELSE sequence_of_statementsN+1;]

END CASE [label_name];



Instrucțiunea CASE

- Ex. 5. Folosirea instrucțiunii searched CASE:

```
set serveroutput on;
declare
  grade float := 7.2;
begin
  case
    when grade >=9 then dbms_output.put_line('excellent');
    when grade >=8 and grade<9 then dbms_output.put_line('very good');
    when grade >=7 and grade<8 then dbms_output.put_line('good');
    when grade >=6 and grade<7 then dbms_output.put_line('fair');
    when grade < 5 then dbms_output.put_line('poor');
    else dbms_output.put_line('no such grade');
  end case;
end;
```

Instrucțiunea CASE

- Ex. 6. Folosirea instrucțiunii CASE pentru inițializarea unei variabile și în clauza where a unei cereri:

```
set serveroutput on;
declare
  alege integer;
  functie emp.job%type;
  nume emp.ename%type;
begin
  alege := &numar;

  -- echivalent
  functie := case
             when alege = 1 then 'SALESMAN'
             when alege = 2 then 'CLERK'
             else 'ANALYST'
           end;
```

```
-- echivalent
functie := case alege
             when '1' then 'SALESMAN'
             when '2' then 'CLERK'
             else 'ANALYST'
           end;
```



Instrucțiunea CASE

```
-- echivalent
select ename into nume from emp
  where empno = (case
                   when functie = 'SALESMAN' then 7844
                   when functie = 'CLERK' then 7900
                   when functie = 'ANALYST' then 7902
                   else 7839
                 end );

  dbms_output.put_line(nume||' are functia '||functie);
end;
```

```
-- echivalent
select ename into nume from emp
  where empno = (case functie
                   when 'SALESMAN' then 7844
                   when 'CLERK' then 7900
                   when 'ANALYST' then 7902
                   else 7839
                 end );
```

Instrucțiunea CASE

- Ex. 7. Exemplu de folosire a instrucțiunii CASE în clauza SELECT.

```
set serveroutput on;
declare
  nume emp.ename%type;
  functie varchar(30);
begin
  select e.ename, (case e.job
                   when 'ANALYST' then 'Analist'
                   when 'SALESMAN' then 'Vanzator'
                   when 'CLERK' then 'Functionar'
                   when 'PRESIDENT' then 'Presedinte'
                   when 'MANAGER' then 'Director'
                   else 'Nu avem functia'
                   end)
    into nume, functie
    from emp e where empno = &idEmp;
  dbms_output.put_line(nume||' are functie '||functie);
end;
```



Instrucțiunea CASE

Observatii:

- Clauza **ELSE** poate să lipsească din instrucțiunea CASE;
- Dacă lipsește clauza **ELSE** și niciuna dintre clauzele **WHEN** nu se execută atunci apare eroarea **CASE_NOT_FOUND.**



Instrucțiunea LOOP

- Instrucțiunea **LOOP** este folosită pentru execuția ciclică a unei secvențe de instrucțiuni;
- Sintaxa:

  [<<label_name>>]

  LOOP

  statements;

  EXIT [label_label] [WHEN conditions];

  END LOOP [label_name];
- **statements** – reprezintă secvența de instrcțiuni executate în cadrul buclei
- **label_name** – este eticheta următoarei instrucțiuni la care se forțează saltul, care poate să lipsească;
- **conditions** – reprezintă setul de condiții la care se forțează ieșirea din buclă și nu este obligatoriu, iar dacă lipsește atunci bucla are o singură iterație.



Instrucțiunea LOOP

- După fiecare procesare a unui ciclu, controlul revine la începutul ciclului și se reia execuția instrucțiunilor;
- Instrucțiunea **EXIT** este folosită pentru ieșirea din ciclu, cu sau fără verificare de condiție;
- **EXIT** poate fi executată fie ca o acțiune dintr-un **IF** în cadrul blocului **LOOP**, fie ca o instrucțiune individuală în cadrul unui ciclu;
- O clauză **WHEN** poate fi înlocuită de instrucțiunea **IF**, cum se vede în construcția următoare:

```
LOOP
        statements;
        IF conditions THEN
                statements_in_if;
                EXIT;  -- exit loop immediately
        END IF;
END LOOP;
```

Instrucțiunea LOOP

- Ex. 8. În exemplul următor se dorește să se facă o listă cu data angajării și veniturile angajaților care fac parte dintr-un departament specificat.

```
set serveroutput on;
declare
   deptId integer;
   contor integer := 7000;
   numeAngajat varchar2(30);
   venit number(10);
   dataAngajare date;
   numeDepartament dept.dname%type;
```

Instrucțiunea LOOP

```
begin
  deptId := &idDept;
  select dname into numeDepartament from dept where deptno=deptId;
  dbms_output.put_line(rpad('Nume', 30,'=')||
    rpad('Data Angajare', 15, '=')||
    lpad('Venit',10,'='));
  loop
    contor := contor + 1;
    begin
      select ename, hiredate, sal+nvl(comm, 0)
        into numeAngajat, dataAngajare, venit
        from emp
        where deptno=deptId and empno=contor;
      dbms_output.put_line(rpad(numeAngajat, 30,'=')||
        rpad(dataAngajare, 15, '=')||
        lpad(venit,10,'='));
    exception -- nu poate fi folosit decat intr-un bloc
      when no_data_found then null;
    end;
    exit when contor=8000;
  end loop;
  exception
      when no_data_found then
        dbms_output.put_line('Nu sunt angajati in departamentul'||deptId);
end;
```

Instrucțiunea WHILE

- Instructiunea **WHILE** este folosită pentru realizarea unei structuri repetitive condiționate;
- Sintaxa instrucțiunii este:

  WHILE conditions

  LOOP

  statements;

  END LOOP;
- **conditions** – reprezintă setul de condiții pentru execuția buclei;
- **statements** – reptezintă secvența de instrucțiuni executată în cadrul buclei.

Instrucțiunea WHILE

- Ex. 9. Să se modifice exercițiul 8 astfel încât să folosească instrucțiunea WHILE.

```
set serveroutput on;
declare
   deptId integer;
   contor integer := 7000;
   numeAngajat varchar2(30);
   venit number(10);
   dataAngajare date;
   numeDepartament dept.dname%type;
```



Instrucțiunea WHILE

```
begin
  deptId := &idDept;
  select dname into numeDepartament from dept where deptno=deptId;
  dbms_output.put_line(rpad('Nume', 30,'=')||
    rpad('Data Angajare', 15, '=')||
    lpad('Venit',10,'='));
  while contor <= 8000
  loop
    contor := contor + 1;
    begin
      select ename, hiredate, sal+nvl(comm, 0)
        into numeAngajat, dataAngajare, venit
        from emp
        where deptno=deptId and empno=contor;
      dbms_output.put_line(rpad(numeAngajat, 30,'=')||
        rpad(dataAngajare, 15, '=')||
        lpad(venit,10,'='));
    exception -- nu poate fi folosit decat intr-un bloc
      when no_data_found then null;
    end;
  end loop;
  exception
      when no_data_found then
        dbms_output.put_line('Nu sunt angajati in departamentul '||deptId);
end;
```



Instrucțiunea FOR

- Instrucțiunea **FOR** este folosită pentru realizarea unei structuri repetitive;
- Sintaxa instrucțiunii este:

  FOR contor IN [REVERSE] value1 .. value2

  LOOP

  statements

  END LOOP;
- **contor** – este contorul folosit pentru incrementarea (decrementarea) iterațiilor
- **value1 .. value2** – reprezintă valorile minime și maxime ale contorului (**value1 <= value2**)
- **statements** – reprezintă secvența de instrucțiuni executate în cadrul buclei



Instrucțiunea FOR

Observatii:

- Incrementarea (cazul cu **IN**) sau decrementarea (cazul **IN REVERSE**) contorului este făcută automat;
- Chiar dacă se folosește **REVERSE** value1 și value2 vor reprezenta tot valorea minimă, respectiv valoarea maximă, diferența apare doar la contor care începe să fie decrementat de la valoarea maximă la valoarea minimă;
- Secvența de valori value1 .. value2 nu poate fi inversată programatic, adica, chiar dacă nu dă nicio eroare, sintaxa următoare nu are niciun rezultat :

```
FOR contor IN maxVal .. minVal
LOOP
        statements
END LOOP
```



Instrucțiunea FOR

- Ex. 10. Să se modifice exercițiul 8 astfel încât să folosească instrucțiunea FOR.

```
set serveroutput on;
declare
   deptId integer;
   contor integer := 7000;
   numeAngajat varchar2(30);
   venit number(10);
   dataAngajare date;
   numeDepartament dept.dname%type;
```



Instrucțiunea FOR

```
begin
  deptId := &idDept;
  select dname into numeDepartament from dept where deptno=deptId;
  dbms_output.put_line(rpad('Nume', 30,'=')||
    rpad('Data Angajare', 15, '=')||
    lpad('Venit',10,'='));
  for contor in 7000 .. 8000
  loop
    begin
      select ename, hiredate, sal+nvl(comm, 0)
        into numeAngajat, dataAngajare, venit
        from emp
        where deptno=deptId and empno=contor;
      dbms_output.put_line(rpad(numeAngajat, 30,'=')||
        rpad(dataAngajare, 15, '=')||
        lpad(venit,10,'='));
    exception -- nu poate fi folosit decat intr-un bloc
      when no_data_found then null;
    end;
  end loop;
  exception
      when no_data_found then
        dbms_output.put_line('Nu sunt angajati in departamentul'||deptId);
end;
```



Instrucțiunea FOR

Observatii:

- Buclele pot fi imbricate pe mai multe niveluri;
- În general, o buclă internă nu termină execuția unui bucle care o include, exceptând cazul când este generată o excepție;
- Pentru salturi de instrucțiuni, se folosește instrucțiunea **GOTO**. Această instrucțiune poate fi folosită în oricare dintre instrucțiunile prezentate;
- Instrucțiunea **GOTO** poate fi folosită și să se iasă dintr-o buclă.



Instrucțiunea FOR

- Ex. 11. Exemplu de folosire a instrucțiunii **GOTO:**

```
set serveroutput on;
declare
  contor integer;
begin
  for contor in 1..10
  loop
    if contor = 5 then goto label1;
    else goto label2;
    end if;
    <<label1>>dbms_output.put_line('Contorul este la jumatate!');
    <<label2>>dbms_output.put_line('Valoarea contorului este '||contor);
  end loop;
end;
```

Instrucțiunea FOR

- Ex. 12. Exeplu de ieșise dintr-o buclă folosind GOTO:

```
set serveroutput on;
declare
  contor integer;
begin
  for contor in 1..10
  loop
    if contor = 6 then goto label1;
    else goto label2;
    end if;
    <<label2>>dbms_output.put_line('Valoarea contorului este '||contor);
  end loop;
  <<label1>>dbms_output.put_line('Am iesit din bucla!');
end;
```

Exercițiu

- Ex. 13. Să se scrie un bloc PL/SQL care face o listă cu numărul angajaților care au veniturile mai mici de 2000 (considerate venituri mici) și mai mari de 2000 (considerate venituri mari), pentru fiecare departament. Se va face o construcție folosind structuri de control și cicluri. Pentru departamentele care nu au niciun angajat se va afișa un mesaj.

```
set serveroutput on;
declare
   contor integer := 0;
   contor1 number;
   contor2 number;
   suma1 number;
   suma2 number;
   numeDepartament dept.dname%type;
```



Exercițiu

```
begin
  dbms_output.put_line(rpad('Nume Departament', 20)||
    rpad('Nr. Sal. Mici',20)||rpad('Suma Sal. Mici',20)||
    rpad('Nr. Sal. Mari',20)||rpad('Suma Sal. Mari',20));
  loop
    begin
      contor := contor + 10;
      select dname into numeDepartament from dept where deptno=contor;
      select count(*), sum(sal+nvl(comm,0)) into contor1, sumal
        from emp where deptno=contor and (sal+nvl(comm,0))<2000;
      select count(*), sum(sal+nvl(comm,0)) into contor2, suma2
        from emp where deptno=contor and (sal+nvl(comm,0))>=2000;
      dbms_output.put_line(rpad(numeDepartament, 20)||
          rpad(contor1,20)||rpad(nvl(sumal,0),20)||
          rpad(contor2,20)||rpad(nvl(suma2,0),20));
      if contor1 = 0 and contor2 = 0 then
        dbms_output.put_line('Departamentul '||numeDepartament
          ||' nu are angajati.');
      end if;
    exception
      when no_data_found then
        if contor >50 then exit;
        else null;
        end if;
    end;
  end loop;
end;
```



Structuri de control PL/SQL

- Mai multe informații găsiți la adresa:
- http://docs.oracle.com/database/121/LNPLS/control statements.htm#LNPLS004